```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

FILE = "/content/drive/MyDrive/Business_Intelligence/BI_Retail_Case_Study.xlsx"

sales    = pd.read_excel(FILE, sheet_name="SalesRaw")
products = pd.read_excel(FILE, sheet_name="DimProduct")
stores   = pd.read_excel(FILE, sheet_name="DimStore")
customers= pd.read_excel(FILE, sheet_name="DimCustomer")
employees= pd.read_excel(FILE, sheet_name="DimEmployee")
targets  = pd.read_excel(FILE, sheet_name="Targets")
dimdate  = pd.read_excel(FILE, sheet_name="DimDate")

sales["TransactionDate"] = pd.to_datetime(sales["TransactionDate"])
dimdate["Date"]          = pd.to_datetime(dimdate["Date"])

# Master merged frame
master = (sales
    .merge(products[["ProductID","ProductName","Category","UnitCost"]], on="ProductID", how="left")
    .merge(stores,   on="StoreID",    how="left")
    .merge(customers[["CustomerID","CustomerType"]], on="CustomerID", how="left")
    .merge(employees[["EmployeeID","EmployeeName","Role"]], on="EmployeeID", how="left")
)
master["GrossProfit"] = master["NetSales"] - (master["UnitCost"] * master["Quantity"])
```

CASE STUDY ONE — Store Performance and Target Achievement

Question 1 Store-Level Analytical Dataset

```
# Actuals by store
store_actual = master.groupby(["StoreID","StoreName","StoreType","Province"]).agg(
    NetSales       = ("NetSales","sum"),
    Transactions   = ("TransactionID","nunique"),
    QuantitySold   = ("Quantity","sum"),
    GrossProfit    = ("GrossProfit","sum"),
).reset_index()

store_actual["ProfitMargin"] = store_actual.GrossProfit / store_actual.NetSales
store_actual["AvgTxnValue"]  = store_actual.NetSales / store_actual.Transactions

# Targets by store
target_store = targets.groupby("StoreID")["SalesTarget"].sum().reset_index()

store_kpi = store_actual.merge(target_store, on="StoreID", how="left")
store_kpi["Achievement%"] = store_kpi.NetSales / store_kpi.SalesTarget * 100
store_kpi["Variance"]     = store_kpi.NetSales - store_kpi.SalesTarget

# Ranks
for c, name in [("NetSales","RankSales"),("GrossProfit","RankProfit"),
                ("ProfitMargin","RankMargin"),("Achievement%","RankAch")]:
    store_kpi[name] = store_kpi[c].rank(ascending=False).astype(int)

print(store_kpi.to_string(index=False))
```

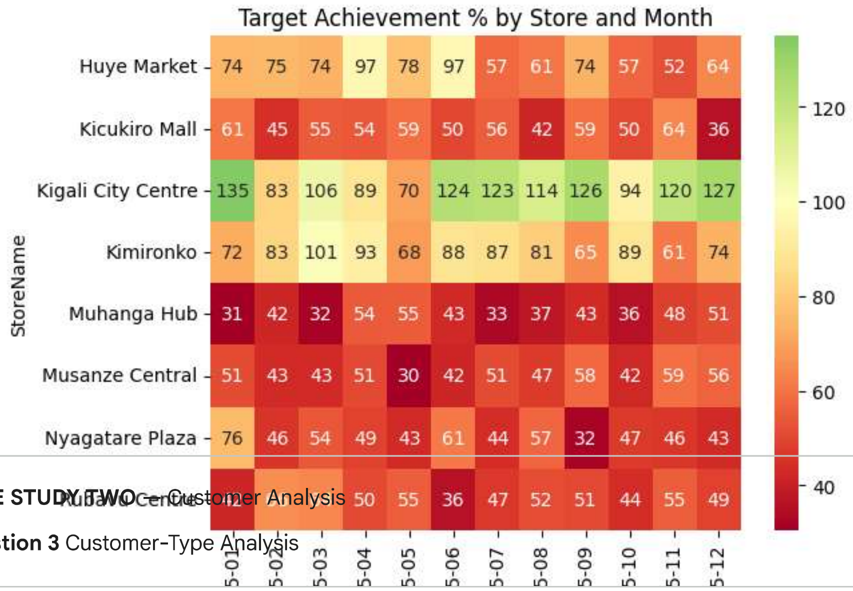
```
 StoreID          StoreName      StoreType Province     NetSales  Transactions  QuantitySold  GrossProfit  ProfitMargin  AvgTxnV
       1 Kigali City Centre          Urban   Kigali 244468330.33          4919         12800  66405280.33      0.271631 49698.78
       2          Kimironko          Urban   Kigali 185077376.01          3976         10058  49545876.01      0.267704 46548.63
       3      Kicukiro Mall          Urban   Kigali 159184431.42          3229          8418  42831581.42      0.269069 49298.36
       4    Musanze Central Secondary City Northern 138977336.14          2739          7174  37267336.14      0.268154 50740.17
       5      Rubavu Centre Secondary City  Western 129660749.23          2421          6293  32994199.23      0.254466 53556.69
       6        Huye Market Secondary City Southern 120250339.92          2543          6668  31806189.92      0.264500 47286.80
       7    Nyagatare Plaza Secondary City  Eastern 132517104.79          2622          6856  35185954.79      0.265520 50540.46
       8        Muhanga Hub Secondary City Southern 124232355.23          2551          6639  33454905.23      0.269293 48699.47
```

Question 2 Monthly Store Performance vs Targets

```
master["Month"] = master.TransactionDate.dt.to_period("M").astype(str)
targets["Month"] = pd.to_datetime(targets["Month"]).dt.to_period("M").astype(str)

monthly = (master.groupby(["Month","StoreID"])["NetSales"].sum()
           .reset_index()
           .merge(targets, on=["Month","StoreID"], how="left")
           .merge(stores[["StoreID","StoreName"]], on="StoreID", how="left"))
monthly["Achievement%"] = monthly.NetSales / monthly.SalesTarget * 100
monthly["Variance"]     = monthly.NetSales - monthly.SalesTarget

# Pivot for heatmap
pivot = monthly.pivot(index="StoreName", columns="Month", values="Achievement%")
sns.heatmap(pivot, annot=True, fmt=".0f", cmap="RdYlGn", center=100)
plt.title("Target Achievement % by Store and Month")
plt.show()
```



CASE STUDY TWO — Customer Analysis

Question 3 Customer-Type Analysis

```
cust = master.groupby("CustomerType").agg(
    TotalSales    = ("NetSales","sum"),
    Transactions  = ("TransactionID","nunique"),
    Quantity      = ("Quantity","sum"),
    GrossProfit   = ("GrossProfit","sum"),
).reset_index()

cust["PctSales"]   = cust.TotalSales / cust.TotalSales.sum() * 100
cust["PctProfit"]  = cust.GrossProfit / cust.GrossProfit.sum() * 100
cust["ATV"]        = cust.TotalSales / cust.Transactions
cust["Margin"]     = cust.GrossProfit / cust.TotalSales * 100

print(cust.to_string(index=False))
```

```
CustomerType   TotalSales  Transactions  Quantity  GrossProfit  PctSales  PctProfit          ATV     Margin
  Individual 892640264.86         18146     47176 238105914.86 72.315569  72.264700 49192.123050 26.674342
 Institution  72290762.33          1271      3350  18461362.33  5.856500   5.602989 56877.075004 25.537651
         SME 269436995.88          5583     14380  72924045.88 21.827931  22.132311 48260.253606 27.065343
```

Question 4 RFM Analysis

```
snapshot = master.TransactionDate.max() + pd.Timedelta(days=1)

rfm = master.groupby("CustomerID").agg(
    Recency   = ("TransactionDate", lambda x: (snapshot - x.max()).days),
    Frequency = ("TransactionID", "nunique"),
    Monetary  = ("NetSales", "sum"),
).reset_index()

# Quintile scores (1 = worst, 5 = best)
rfm["R"] = pd.qcut(rfm.Recency, 5, labels=[5,4,3,2,1]).astype(int)
rfm["F"] = pd.qcut(rfm.Frequency.rank(method="first"), 5, labels=[1,2,3,4,5]).astype(int)
rfm["M"] = pd.qcut(rfm.Monetary.rank(method="first"), 5, labels=[1,2,3,4,5]).astype(int)
rfm["RFM_Score"] = rfm.R + rfm.F + rfm.M

def segment(row):
    if row.R >= 4 and row.F >= 4 and row.M >= 4: return "Champions"
    if row.R >= 3 and row.F >= 3:                return "Loyal"
    if row.R >= 4 and row.F <= 2:                return "New/Promising"
    if row.R <= 2 and row.F >= 3:                return "At Risk"
    if row.R <= 2 and row.F <= 2:                 return "Lost"
    return "Other"

rfm["Segment"] = rfm.apply(segment, axis=1)
print(rfm.Segment.value_counts())
print(rfm.groupby("Segment")[["Recency","Frequency","Monetary"]].mean().round(1))
```

```
Segment
Loyal           1405
Lost            1089
At Risk          896
Champions        684
New/Promising    558
Other            343
Name: count, dtype: int64
               Recency  Frequency  Monetary
Segment
At Risk          111.5        6.0  301417.2
Champions         18.1        7.7  501265.4
Lost             155.5        2.8  120463.2
Loyal             33.5        6.0  244970.6
New/Promising     19.0        3.3  165178.8
Other             53.2        3.2  157128.0
```

CASE STUDY THREE — Product Profitability and Discount Strategy

Question 5 Product and Category Performance

```
cat = master.groupby("Category").agg(
    NetSales       = ("NetSales","sum"),
    Quantity       = ("Quantity","sum"),
    GrossProfit    = ("GrossProfit","sum"),
    AvgDiscount    = ("DiscountPct","mean"),
    NumProducts    = ("ProductID","nunique"),
).reset_index()

cat["Margin"]    = cat.GrossProfit / cat.NetSales * 100
cat["PctSales"]  = cat.NetSales / cat.NetSales.sum() * 100
cat = cat.sort_values("NetSales", ascending=False)
print(cat.to_string(index=False))
```

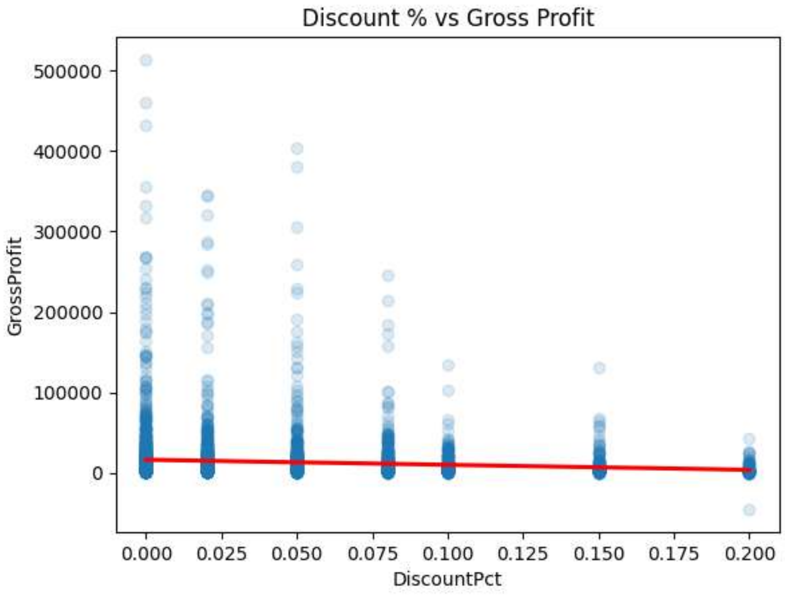
```
          Category     NetSales  Quantity  GrossProfit  AvgDiscount  NumProducts    Margin  PctSales
       Electronics 648827368.50     10762 138824268.50     0.040942           10 21.396180 52.563527
    Home & Kitchen 251552603.03     11075  85044903.03     0.042037           10 33.808000 20.379060
       Agriculture 112977816.50     10401  35516116.50     0.041753           10 31.436363  9.152685
     Personal Care  99739597.91     11047  32576597.91     0.042336           10 32.661650  8.080216
Office & Stationery  91296939.80     10858  29794439.80     0.042414           10 32.634653  7.396250
    Food & Beverage  29973697.33     10763   7734997.33     0.042234           10 25.805950  2.428263
```

Question 6 Discount vs Profitability

```
# Scatter with trend
sns.regplot(data=master.sample(5000), x="DiscountPct", y="GrossProfit",
            scatter_kws={"alpha":0.15}, line_kws={"color":"red"})
plt.title("Discount % vs Gross Profit")
plt.show()

# Correlation
print(master[["DiscountPct","GrossProfit"]].corr())

# Banded averages
master["DiscBand"] = pd.cut(master.DiscountPct,
    bins=[-0.001,0,0.05,0.10,0.15,0.20,1],
    labels=["0%","0-5%","5-10%","10-15%","15-20%",">20%"])
print(master.groupby("DiscBand")[["GrossProfit","NetSales"]].mean().round(2))
```



```
             DiscountPct  GrossProfit
DiscountPct     1.000000    -0.077834
GrossProfit    -0.077834     1.000000
          GrossProfit  NetSales
DiscBand
0%           15418.05  51866.20
0-5%         13274.16  48324.76
5-10%        11110.50  48539.57
10-15%        7819.94  46774.17
15-20%        4811.53  38391.00
>20%              NaN       NaN
/tmp/ipykernel_375/1300588836.py:14: FutureWarning: The default of observed=False is deprecated and will be changed to True in a
  print(master.groupby("DiscBand")[["GrossProfit","NetSales"]].mean().round(2))
```

CASE STUDY FOUR — Sales Channel and Operational Performance

Question 7 Sales Channel Comparison

```
channel = master.groupby("StoreType").agg(
    Transactions = ("TransactionID","nunique"),
    NetSales     = ("NetSales","sum"),
    GrossProfit  = ("GrossProfit","sum"),
).reset_index()

channel["Margin"]  = channel.GrossProfit / channel.NetSales * 100
channel["ATV"]     = channel.NetSales / channel.Transactions
print(channel.to_string(index=False))

# If a proper channel column exists, replace "StoreType" with that.
```

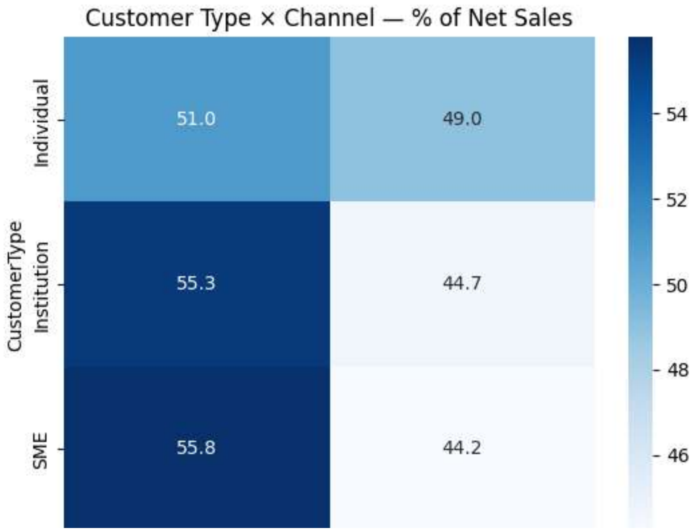
```
     StoreType  Transactions     NetSales  GrossProfit    Margin          ATV
Secondary City         12876 645637885.31 170708585.31 26.440299 50142.737287
         Urban         12124 588730137.76 158782737.76 26.970377 48559.067780
```

Question 8 Customer Type × Sales Channel Cross-Tab

```
crosstab = pd.crosstab(master.CustomerType, master.StoreType,
                       values=master.NetSales, aggfunc="sum", normalize="index") * 100

sns.heatmap(crosstab, annot=True, fmt=".1f", cmap="Blues")
plt.title("Customer Type × Channel — % of Net Sales")
plt.show()

print(crosstab.round(2))
```


Customer Type × Channel — % of Net Sales
CustomerType
Individual
Institution
SME
51.0
49.0
55.3
44.7
55.8
44.2
54
52
50
48
46